

“Instituto Politécnico Nacional”

“Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas”



Alumno: Ortiz Campos Kevin Daniel.

Grupo: 4TM3.

Escuela: Unidad Profesional
Interdisciplinaria en
Ingeniería y Tecnologías
Avanzadas.

Materia: Aplicaciones Distribuidas.

Profesor: Sierra Romero Noé.

Trabajo: Practica 07.

Practica 7 MongoDB

Práctica sobre la inserción de registros de la Base de datos de AWS en mongo DB:

Usuario: kdocdany_db_user

Contraseña: IBIUz597KFx5jQwG

Comandos utilizados:

Init -d

npm install mongodb

npm install express

URI:

mongodb+srv://kdocdany_db_user:IBIUz597KFx5jQwG@cluster0.u4w7qod.mongodb.net/?appName=Cluster0

```
// npm install express

require("node:dns/promises").setServers(["1.1.1.1", "8.8.8.8"]);

var express = require("express");

var app = express(); //Contenedor de Endpoints o WS Restful

const { MongoClient } = require("mongodb");

var client = 0;


var dbName = "";

var collectionName = "";


// Create references to the database and collection in order to run

// operations on them.
```

```
var database = 0;

var collection = 0;


app.use(express.json());
app.use(express.urlencoded({ extended: true }));


function prepareDB() {
  dbName = "myDatabase";
  collectionName = "recipes";

  // Create references to the database and collection in order to run
  // operations on them.
  database = client.db(dbName);
  collection = database.collection(collectionName);
}


async function connectDB() {
  const uri =

    "mongodb+srv://kdocdany_db_user:IBIUz597KFx5jQwG@cluster0.u4w7qod.mongodb.net/?appName=Cluster0";

  client = new MongoClient(uri);

  // The connect() method does not attempt a connection; instead it instructs
  // the driver to connect using the settings provided when a connection
  // is required.
  await client.connect();
}
```

```
app.get("/", async function (request, response) {
```

```
  r = {
```

```
    message: "Nothing to send",
```

```
  };
```

```
  response.json(r);
```

```
});
```

```
/*
```

Calling this service sending payload as parameters in URL:

<https://typesofwebservices.noesierra.repl.co/serv001?id=Nope&token=2345678dhuj43567fgh&geo=123456789,1234567890>

```
*/
```

```
app.get("/serv001", async function (req, res) {
```

```
  const user_id = req.query.id;
```

```
  const token = req.query.token;
```

```
  const geo = req.query.geo;
```

```
  r = {
```

```
    user_id: user_id,
```

```
    token: token,
```

```
    geo: geo,
```

```
  };
```

```
  res.json(r);
```

```
});
```

```
/*
```

Calling this service sending payload as parameters in URL:

<https://typesofwebservices.noesierra.repl.co/serv001?id=Nope&token=2345678dhuj43567fgh&geo=123456789,1234567890>

```
*/
```

```
app.get("/serv0010", async function (req, res) {
```

```
  const user_id1 = req.query.id;
```

```
  const token1 = req.query.token;
```

```
  const geo1 = req.query.geo;
```

```
  r1 = {
```

```
    user_id: user_id1,
```

```
    token: token1,
```

```
    geo: geo1,
```

```
  };
```

```
  res.json(r1);
```

```
});
```

// Call this service sending payload in body: raw - json

```
/*
```

```
{
```

```
  "id": "nope",
```

```
  "token": "ertydfg456Dfgwerty",
```

```
  "geo": "12345678,34567890"
```

```
}
```

```
*/
```

```
app.post("/serv002", async function (req, res) {
```

```
const user_id = req.body.id;

const token = req.body.token;

const geo = req.body.geo;


r = {
  user_id: user_id,
  token: token,
  geo: geo,
};

res.json(r);
});


/*
Call this service sending parameter as a part of the URL
https://typesofwebservices.noesierra.repl.co/serv003/1234567
*/

app.post("/serv003/:info", async function (req, res) {
  const info = req.params.info;

  let r = { info: info };

  res.json(r);
});


app.post("/receipt/insert", async function (req, res) {
  const recipes = [
    {
      name: "elotes cocidos",
```

```
ingredients: [
  "corn",
  "mayonnaise",
  "cotija cheese",
  "sour cream",
  "lime",
],
prepTimeInMinutes: 35,
},
];

let result = "";

try {
  const insertManyResult = await collection.insertMany(recipes);
  console.log(
    `${insertManyResult.insertedCount} documents successfully inserted.\n`,
  );
  result = `${insertManyResult.insertedCount} documents successfully inserted.`;
} catch (err) {
  console.error(
    `Something went wrong trying to insert the new documents: ${err}\n`,
  );
  result = `Something went wrong trying to insert the new documents: ${err}`;
}

let r = { result: result };

res.json(r);
```

```
});

app.listen(3000, async function () {

  console.log("Aplicación ejemplo, escuchando el puerto 3000!");

  try {

    await connectDB(); // Esperamos a que conecte

    prepareDB();      // Ahora sí preparamos la DB

    console.log("Conexión a MongoDB exitosa");

  } catch (e) {

    console.error("Error al conectar a MongoDB:", e);

  }

});
```

```
Administrador: Símbolo del sistema
Microsoft Windows [Versión 10.0.22631.6199]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>cd D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB

C:\Windows\System32>D:

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>npm init -d
npm info using npm@11.6.2
npm info using node@v24.13.0
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (proyecto7_insertmongodb)
version: (1.0.0)
description: Servicio que inserta recetas
entry point: (insertMongoDB.js)
test command:
git repository:
keywords:
author: Daniel OC
license: (ISC)
type: (commonjs)
About to write to D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB\package.json:

{
  "name": "proyecto7_insertmongodb",
  "version": "1.0.0",
  "description": "Servicio que inserta recetas",
  "main": "insertMongoDB.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Daniel OC",
  "license": "ISC",
  "type": "commonjs"
}
```

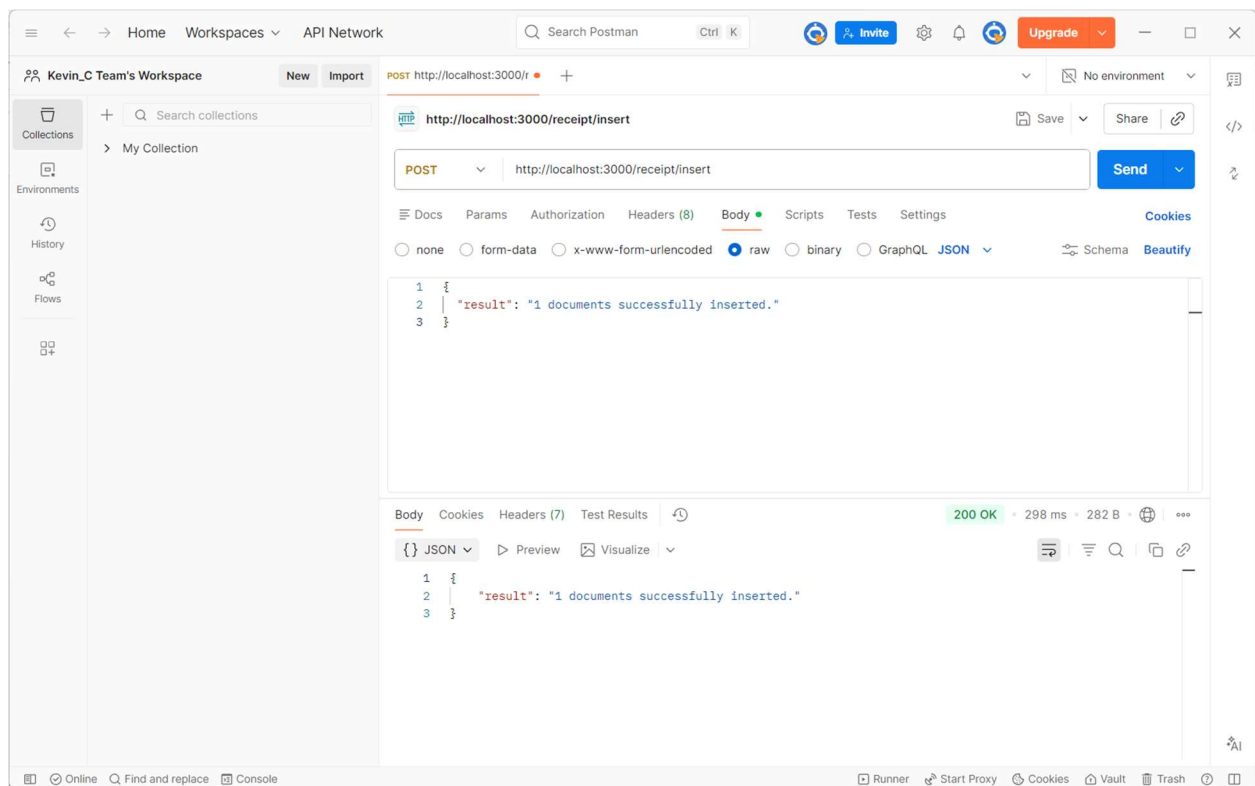
```
Is this OK? (yes)
```

```
npm info ok
```

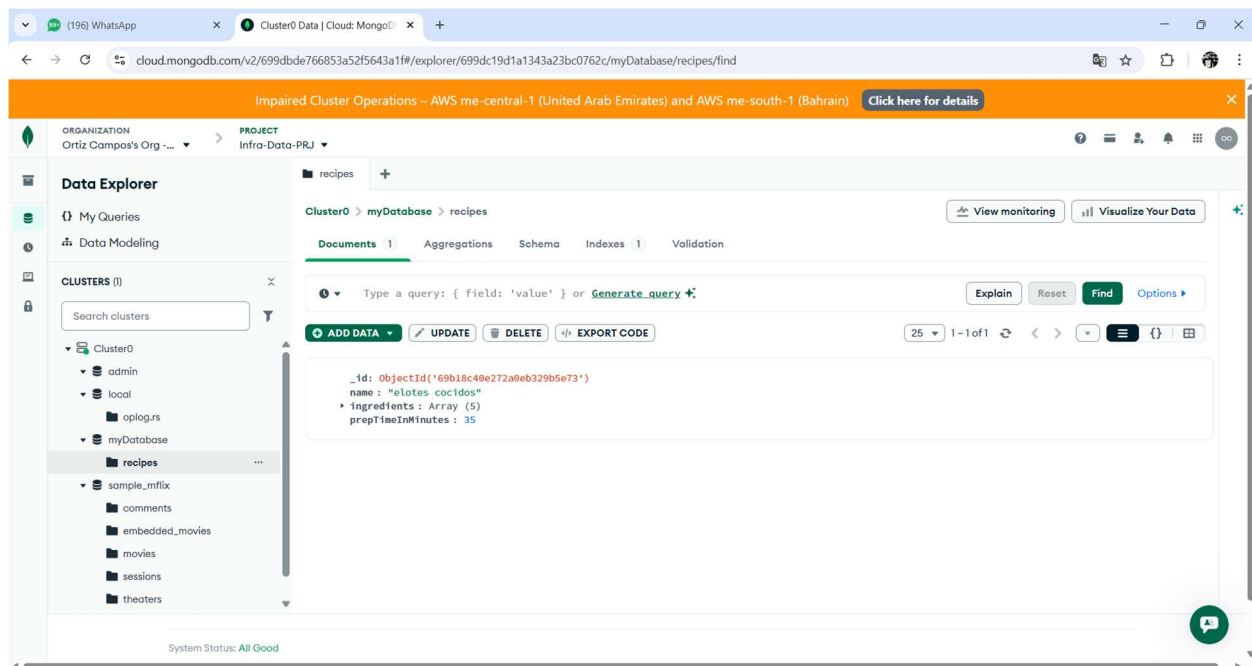
```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>npm install mongodb
added 12 packages, and audited 13 packages in 13s
found 0 vulnerabilities
```

```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>npm install express
added 65 packages, and audited 78 packages in 41s
22 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>node insertMongoDB.js
Aplicación ejemplo, escuchando el puerto 3000!
Conexión a MongoDB exitosa
```



```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>node insertMongoDB.js
Aplicación ejemplo, escuchando el puerto 3000!
Conexión a MongoDB exitosa
1 documents successfully inserted.
```



Agregando parámetros

```
// npm install express

require("node:dns/promises").setServers(["1.1.1.1", "8.8.8.8"]);

var express = require("express");

var app = express(); //Contenedor de Endpoints o WS Restful

const { MongoClient } = require("mongodb");

var client = 0;

var dbName = "";

var collectionName = "";

// Create references to the database and collection in order to run
// operations on them.

var database = 0;

var collection = 0;
```

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

function prepareDB() {
  dbName = "myDatabase";
  collectionName = "recipes";

  // Create references to the database and collection in order to run
  // operations on them.
  database = client.db(dbName);
  collection = database.collection(collectionName);
}

async function connectDB() {
  const uri =
    "mongodb+srv://kdocdany_db_user:IBIUz597KFx5jQwG@cluster0.u4w7qod.mongodb.net/?appName=Cluster0";

  client = new MongoClient(uri);

  // The connect() method does not attempt a connection; instead it instructs
  // the driver to connect using the settings provided when a connection
  // is required.
  await client.connect();
}

app.get("/", async function (request, response) {
```

```
    r = {  
      message: "Nothing to send",  
    };  
  };  
}
```

```
    response.json(r);  
  });  
}
```

```
/*
```

Calling this service sending payload as parameters in URL:

<https://typesofwebservices.noesierra.repl.co/serv001?id=Nope&token=2345678dhuj43567fgh&geo=123456789,1234567890>

```
*/
```

```
app.get("/serv001", async function (req, res) {
```

```
  const user_id = req.query.id;
```

```
  const token = req.query.token;
```

```
  const geo = req.query.geo;
```

```
  r = {  
    user_id: user_id,  
    token: token,  
    geo: geo,  
  };  
}
```

```
  res.json(r);  
});  
}
```

```
/*
```

Calling this service sending payload as parameters in URL:

<https://typesofwebservices.noesierra.repl.co/serv001?id=Nope&token=2345678dhuj43567fgh&geo=123456789,1234567890>

*/

```
app.get("/serv0010", async function (req, res) {
```

```
  const user_id1 = req.query.id;
```

```
  const token1 = req.query.token;
```

```
  const geo1 = req.query.geo;
```

```
  r1 = {
```

```
    user_id: user_id1,
```

```
    token: token1,
```

```
    geo: geo1,
```

```
  };
```

```
  res.json(r1);
```

```
});
```

// Call this service sending payload in body: raw - json

/*

{

```
  "id": "nope",
```

```
  "token": "ertydfg456Dfgwerty",
```

```
  "geo": "12345678,34567890"
```

}

*/

```
app.post("/serv002", async function (req, res) {
```

```
  const user_id = req.body.id;
```

```
  const token = req.body.token;
```

```
const geo = req.body.geo;

r = {
  user_id: user_id,
  token: token,
  geo: geo,
};

res.json(r);
});

/*
Call this service sending parameter as a part of the URL
https://typesofwebservices.noesierra.repl.co/serv003/1234567
*/

app.post("/serv003/:info", async function (req, res) {
  const info = req.params.info;

  let r = { info: info };

  res.json(r);
});

app.post("/receipt/insert", async function (req, res) {
  const recipes = req.body;

  if (!recipes || Object.keys(recipes).length === 0) {
    return res.status(400).json({ result: "Error: El cuerpo de la petición está vacío." });
  }
}
```

```
let result = "";

try {

  const recipesToInsert = Array.isArray(recipes) ? recipes : [recipes];

  const insertResult = await collection.insertMany(recipesToInsert);

  console.log(`${insertResult.insertedCount} documento(s) insertado(s).\n`);
  result = `${insertResult.insertedCount} documento(s) insertado(s) correctamente.`;

  res.json({ result: result });

} catch (err) {

  console.error(`Error al insertar: ${err}\n`);
  res.status(500).json({ result: `Error interno: ${err.message}` });

}

});

app.listen(3000, async function () {

  console.log("Aplicación ejemplo, escuchando el puerto 3000!");

  try {

    await connectDB(); // Esperamos a que conecte

    prepareDB();      // Ahora sí preparamos la DB

    console.log("Conexión a MongoDB exitosa");

  } catch (e) {

    console.error("Error al conectar a MongoDB:", e);

  }

});
```

```
});
```

^C

```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto7_insertMongoDB>node insertMongoDB.js
Aplicación ejemplo, escuchando el puerto 3000!
Conexión a MongoDB exitosa
1 documento(s) insertado(s).
```

The screenshot shows the Postman application interface. On the left, the 'Collections' sidebar is visible with 'My Collection' selected. The main workspace displays a POST request to 'http://localhost:3000/receipt/insert'. The request body is a JSON object:

```
{ 1: { 2: 'name': 'Enchiladas Verdes', 3: 'ingredients': ['tortillas', 'salsa verde', 'pollo', 'crema', 'queso oaxaca'], 4: 'prepTimeInMinutes': 30 5: } }
```

. The response tab shows a '200 OK' status with a response body:

```
{ 1: { 2: 'result': '1 documento(s) insertado(s) correctamente.' 3: } }
```

. The bottom status bar indicates '200 OK', '256 ms', and '290 B'.

The screenshot shows the MongoDB Cloud Data Explorer interface. The top navigation bar includes 'Data Explorer', 'My Queries', 'Data Modeling', and 'CLUSTERS (0)'. The main view displays the 'recipes' collection in the 'myDatabase' database. The 'Documents' tab is active, showing a list of documents. The first document is expanded, showing its structure:

```
{ 0: 'corn' 1: 'mayonnaise' 2: 'cotija cheese' 3: 'sour cream' 4: 'lime' prepTimeInMinutes: 35 }
```

. The second document is also expanded, showing its structure:

```
{ _id: ObjectId('69b194da908103f7af09dc82') name: 'Enchiladas Verdes' ingredients: Array (5) 0: 'tortillas' 1: 'salsa verde' 2: 'pollo' 3: 'crema' 4: 'queso oaxaca' prepTimeInMinutes: 30 }
```

. The bottom status bar indicates 'System Status: All Good' and '©2024 MongoDB, Inc.'.